GeoClaw-claude

Python GIS Analysis Toolkit

User Guide

Version 2.2.1 | March 2025

UrbanComp Lab

https://urbancomp.net

MIT License | Copyright © 2025 UrbanComp Lab

1. Project Overview

GeoClaw-claude is a lightweight Python geospatial analysis toolkit developed by UrbanComp Lab. Modeled after the QGIS Processing Framework, it focuses on urban spatial data analysis and visualization with a clean, Pythonic API and a full-featured command-line interface.

Starting from v1.3.0, GeoClaw-claude introduces a Natural Language (NL) operation system, enabling users to drive GIS analysis directly with natural language commands — no API memorization required. Since v2.1.0, the toolkit integrates trackintel for human mobility analysis, supporting GPS trajectory processing from raw positionfixes to semantic locations.

1.1 Core Capabilities

Capability	Description
Natural Language GIS	Drive spatial analysis with natural language commands (CLI + Python API)
Human Mobility Analysis	GPS trajectory processing based on trackintel: staypoints, triplegs, trips, locations
Spatial Analysis	Buffer, clip, nearest neighbor, KDE density estimation
Network Analysis	Shortest path, isochrone, service area, network statistics
Raster Analysis	DEM slope/aspect, reclassification, raster calculator, zonal statistics
Data I/O	GeoJSON/SHP, OSM download, HTTP/WFS remote data, Tianditu API
Cartography	Static maps (4 themes), Folium interactive maps, multi-layer layout
Coordinate Transform	Full WGS84 ↔ GCJ-02 ↔ BD-09 conversion
Memory System	Short-term (session cache) + Long-term (persistent knowledge base)
Auto-update	Version detection, self-check health report, one-command upgrade

1.2 Version History

Version	Changes
v2.2.1 (2025-03)	README restructured; mobility chapter enhanced with NL

Version	Changes
	examples and full demo; NL keyword mapping fixes
v2.2.0 (2025-03)	Wuhan GPS trajectory demo dataset (37,549 points); full demo script; trackintel algorithm attribution
v2.1.0 (2025-03)	analysis/mobility/ module (trackintel integration); 10 NL mobility operations; 20/20 tests pass
v2.0.0 (2025-03)	Major release: NL-driven GIS platform; full documentation sync; API fully backward compatible
v1.3.0 (2025-03)	Natural Language operation system (NLProcessor/NLExecutor/GeoAgent); ask and chat CLI commands
v1.2.0 (2025-03)	Self-check & auto-update (check/update/self-check); health report
v1.1.0 (2025-03)	Memory system (ShortTerm + LongTerm + MemoryManager); memory CLI command group
v1.0.0 (2025-03)	First release: CLI, Skill system, network/raster analysis, coordinate transform

2. Installation & Configuration

2.1 Requirements

Package	Purpose
Python	3.9 or later
geopandas / shapely	Core vector data processing
pyproj / rasterio	Projection and raster I/O
networkx / osmnx	Network construction and analysis
trackintel ≥ 1.4.2	Human mobility analysis (GPS trajectory processing)
scipy	KDE density estimation and statistics
matplotlib / folium	Static and interactive visualization
click	Command-line interface framework
anthropic	Claude AI API (optional; required for NL AI mode and Skill AI calls)

2.2 Quick Install

Clone the repository and run the install script:

```
git clone https://github.com/whuyao/GeoClaw_Claude.git
cd GeoClaw_Claude
bash install.sh
```

Or install manually with pip:

```
pip install -e .          # Development mode (edits take effect
immediately)
pip install trackintel geopandas osmnx anthropic
```

2.3 Initialization

After installation, run the onboarding wizard to configure API keys, data directories, and proxy settings:

```
geoclaw-claude onboard
```

Configuration is saved to ~/.geoclaw_claude/config.json. You can also set individual values:

```
geoclaw-claude config set anthropic_api_key YOUR_KEY
geoclaw-claude config show
```

> 💡 The Anthropic API Key is optional. Without it, the NL system automatically falls back to rule-based mode, which supports all basic GIS operations offline.

3. Natural Language Operation System

GeoClaw-claude v1.3.0+ allows driving GIS analysis entirely with natural language — no API memorization required. The NL system supports 30+ operation types across two modes: AI mode (uses Claude API for semantic understanding) and Rule mode (keyword + regex, fully offline).

3.1 How It Works

Component	Role
NLProcessor	Parses natural language into structured ParsedIntent. Automatically selects AI mode (if API key available) or Rule mode (offline fallback).
NLExecutor	Maps ParsedIntent to actual geoclaw_claude analysis functions. Manages layer context across turns.
GeoAgent	Multi-turn dialog agent. Maintains layer context across conversation. Automatically writes operations to Memory.

3.2 CLI Usage

Single instruction:

```
geoclaw-claude ask "Buffer hospitals by 1km"
geoclaw-claude ask "Download Wuhan park data"
geoclaw-claude ask "Load hospitals.geojson then buffer 500m then
visualize"
```

Debug mode — parse intent only, do not execute:

```
geoclaw-claude ask --dry-run "KDE density analysis on hospitals"
```

Force rule mode (offline) or AI mode:

```
geoclaw-claude ask --rule "Clip hospitals to boundary"
geoclaw-claude ask --ai  "Overlay hospitals and metro stations"
```

Interactive multi-turn dialog (layer context persists across turns):

```
geoclaw-claude chat
geoclaw-claude chat --ai
```

3.3 Python API

```
from geoclaw_claude.nl import GeoAgent

agent = GeoAgent()   # auto-selects AI / rule mode

agent.chat("Load data/wuhan/hospitals.geojson")
```

```
# ✓ Loaded 200 features  Duration: 0.12s

agent.chat("Buffer hospitals by 1km")
# ✓ Buffer complete, 200 features, radius 1000.0m  Duration: 0.43s

agent.chat("Show result on interactive map")
# ✓ Map generated

agent.print_history()
agent.end(title="Wuhan Hospital Analysis")  # flush to long-term memory
```

💡 When confidence is below 60%, the agent asks for confirmation before executing. Reply yes / 是 to proceed.

3.4 Supported Operations (30+)

Operation Type	Example Input
Data loading	"Load hospitals.geojson" / "Download Wuhan park data"
Spatial analysis	"Buffer hospitals by 1km" / "Clip hospitals to boundary"
Statistical analysis	"KDE density analysis on hospitals" / "Zonal statistics by district"
Network analysis	"Shortest path between two points" / "10-minute walk isochrone"
Coordinate transform	"Convert hospitals from wgs84 to gcj02"
Mobility: load data	"Load gps_tracks.csv" / "Read positionfixes"
Mobility: staypoints	"Generate staypoints" / "Staypoint detection"
Mobility: full hierarchy	"One-click mobility analysis" / "Full mobility hierarchy"
Mobility: transport mode	"Predict transport mode" / "Mode prediction"
Mobility: locations	"Identify important locations" / "Identify home and work"
Mobility: metrics	"Mobility metrics summary" / "Radius of gyration"
Mobility: visualization	"Trajectory map" / "Activity heatmap" / "Modal split chart"
Visualization	"Visualize current result" / "Show on interactive map"
Multi-step pipeline	"Load data then buffer then visualize"

4. Human Mobility Analysis (trackintel)

The analysis/mobility/ module integrates the trackintel open-source framework from ETH Zurich, providing a complete pipeline from raw GPS trajectories to mobility metrics and semantic location identification.

> ⚠️ Algorithm Attribution: The core trajectory processing algorithms in this module originate from trackintel (https://github.com/mie-lab/trackintel), developed by the Mobility Information Engineering Lab at ETH Zurich. See Section 4.5 for the full citation.

4.1 Data Hierarchy Model

The trackintel data hierarchy transforms raw GPS points into semantically meaningful mobility layers:

Layer	Description
positionfixes	Raw GPS trajectory points — the starting point for all analysis
staypoints	Locations where a user stayed for a minimum time (e.g., 5 min within 80m). Detected via sliding window with spatial + temporal thresholds.
triplegs	Continuous movement segments between staypoints. Each tripleg can be labeled with a transport mode (walk/bike/car/train).
trips	Complete journeys between activity staypoints, aggregating triplegs into meaningful travel episodes.
locations	Semantically important places (home, work, frequent venues) identified by DBSCAN clustering of staypoints.

4.2 Natural Language Usage (CLI)

Step-by-step analysis:

```
# Step 1: Load GPS data
geoclaw-claude ask "Load data/mobility/wuhan_gps_tracks.csv"

# Step 2: Generate staypoints (default: 80m distance, 5min time threshold)
geoclaw-claude ask "Generate staypoints"

# Step 3: Generate triplegs and predict transport mode
geoclaw-claude ask "Generate triplegs"
geoclaw-claude ask "Predict transport mode"

# Step 4: Generate locations and identify home/work
geoclaw-claude ask "Identify important locations"
geoclaw-claude ask "Identify home and work"
```

```
# Visualize
geoclaw-claude ask "Trajectory map"
geoclaw-claude ask "Activity heatmap"
geoclaw-claude ask "Modal split chart"
geoclaw-claude ask "Mobility metrics summary"
```

Or complete the entire analysis pipeline in one command:

```
geoclaw-claude ask "One-click mobility analysis"

# Or as a full pipeline
geoclaw-claude ask "Load gps_tracks.csv then one-click mobility analysis
then trajectory map"
```

4.3 Python API

```
from geoclaw_claude.analysis.mobility import (
    read_positionfixes, generate_full_hierarchy,
    identify_home_work, mobility_summary,
    plot_mobility_layers, plot_activity_heatmap,
    plot_modal_split, plot_mobility_metrics,
)

# 1. Load GPS data (CSV / GeoDataFrame / DataFrame)
pfs = read_positionfixes(
    "data/mobility/wuhan_gps_tracks.csv",
    user_id_col="user_id",
    tracked_at_col="tracked_at",
    lon_col="longitude",
    lat_col="latitude",
)

# 2. Generate full hierarchy in one call
hierarchy = generate_full_hierarchy(
    pfs,
    dist_threshold=80,      # staypoint: within 80m
    time_threshold=5,       # staypoint: at least 5 min
    location_epsilon=120,   # location clustering: 120m radius
    predict_mode=True,      # auto-predict transport mode
)
# hierarchy keys: positionfixes / staypoints / triplegs / trips /
locations

# 3. Metrics
summary = mobility_summary(hierarchy)
# returns: n_users, n_staypoints, radius_of_gyration_m, jump_length_m, ...

# 4. Identify home/work (OSNA method)
locs = identify_home_work(
```

```
    hierarchy["staypoints"], hierarchy["locations"], method="osna"
)

# 5. Visualization
plot_mobility_layers(hierarchy, save_path="mobility_map.png")
plot_activity_heatmap(hierarchy["staypoints"], save_path="heatmap.png")
plot_modal_split(hierarchy["triplegs"], save_path="modal.png")
plot_mobility_metrics(summary, save_path="dashboard.png")
```

4.4 Demo Dataset & Demo Script

The data/mobility/ directory provides a ready-to-use Wuhan urban GPS trajectory dataset:

Item	Details
File	wuhan_gps_tracks.csv + users_meta.csv
Scale	37,549 GPS points, 5 users, 10 days (2024-01-15 to 2024-01-25)
Coverage	Wuhan Three Towns (Hankou / Wuchang / Hanyang)
User Profiles	Finance, IT Engineer, Trade, Researcher, Healthcare — various commute modes
Transport Modes	Metro, Bus, Walk, Bike — reflecting real Wuhan commuting patterns
Typical Results	~1,781 staypoints, ~1,240 triplegs, ~20 locations (default parameters)

Run the full demo (7-step analysis + 5 output charts):

```
python examples/wuhan_mobility_demo.py
```

Output File	Content
01_mobility_layers_map.png	Layered mobility map: staypoints + triplegs (colored by mode) + locations
02_modal_split.png	Transport mode composition: pie chart + bar chart
03_activity_heatmap_all.png	All-user activity heatmap (Weekday × Hour matrix)
03b_activity_heatmap_u0.png	Single-user (Finance) activity heatmap
04_mobility_metrics_dashboard.pn g	Metrics dashboard: radius of gyration, jump length, modal split, tracking quality

Output File	Content
05_user1_trajectory.png	Single-user (IT Engineer, Optics Valley) personal trajectory map

4.5 Available Mobility Metrics

Function	Description
radius_of_gyration()	Radius of gyration (km) — measures spatial range of activity
jump_lengths()	Jump length distribution (km) — single-trip distance statistics
modal_split()	Transport mode share by count, duration, or distance
tracking_quality()	Temporal coverage ratio (0~1) — data completeness indicator
identify_home_work()	Semantic labeling of home and work locations (OSNA or frequency method)
mobility_summary()	Comprehensive summary aggregating all core metrics

4.6 Algorithm Attribution: trackintel

The core trajectory processing algorithms in geoclaw_claude/analysis/mobility/ originate from trackintel:

Item	Details
GitHub	https://github.com/mie-lab/trackintel
Team	Mobility Information Engineering Lab, ETH Zurich (mie-lab.ethz.ch)
Version required	trackintel >= 1.4.2
License	MIT License

Citation:

> Martin, H., Hong, Y., Wiedemann, N., Bucher, D., & Raubal, M. (2023). Trackintel: An open-source Python library for human mobility analysis. *Computers, Environment and Urban Systems*, 101, 101938. https://doi.org/10.1016/j.compenvurbsys.2023.101938

GeoClaw-claude contributes: unified API wrapping, GeoLayer ecosystem integration, natural language operation interface, and UrbanComp Lab-styled visualization. Original algorithm implementations belong to trackintel and its authors.

5. Memory System

The Memory system provides cross-session knowledge accumulation, enabling GeoClaw-claude to learn from past analyses. It consists of two tiers:

5.1 Short-Term Memory (Session Cache)

In-memory storage scoped to the current session. Thread-safe with RLock. Supports TTL (time-to-live), operation logging, and context key-value storage.

```
from geoclaw_claude.memory import get_memory

mem = get_memory()
mem.start_session("wuhan_hospital_analysis")

# Cache intermediate results
mem.remember("buf_hospitals", buf_layer)
layer = mem.recall_short("buf_hospitals")

# Log operations
mem.log_op("buffer", "hospitals, 1km")

# Set session context
mem.set_context("city", "wuhan")
```

5.2 Long-Term Memory (Persistent)

Persisted to disk at ~/.geoclaw_claude/memory/. Supports keyword search, importance scoring, and tagging. Entries survive across sessions.

```
# Store knowledge
mem.learn(
    title="Wuhan Hospital Spatial Distribution",
    content={"finding": "Hospitals concentrated within 3rd Ring Road"},
    tags=["wuhan", "hospital", "spatial"],
    importance=0.8,
)

# Retrieve
results   = mem.recall("wuhan hospital")       # keyword search
recent    = mem.recall_recent(n=5)              # most recent
important = mem.recall_important(n=5, threshold=0.7)  # by importance

# End session — auto-flush to long-term memory
mem.end_session(title="Wuhan Hospital Coverage Analysis", importance=0.8)
```

5.3 CLI Commands

Command	Description
geoclaw-claude memory status	Show memory system status and statistics
geoclaw-claude memory list	List all long-term memory entries
geoclaw-claude memory list -c knowledge	Filter by category
geoclaw-claude memory search "wuhan hospital"	Keyword search
geoclaw-claude memory learn "title" "content"	Manually add a knowledge entry
geoclaw-claude memory forget <id>	Delete a memory entry
geoclaw-claude memory compact	Compress old memories
geoclaw-claude memory export -o backup.json	Export all memories to JSON

6. Auto-Update & Self-Check

GeoClaw-claude includes a built-in update and health monitoring system. It checks the GitHub repository for new versions, performs environment validation, and generates a comprehensive health report.

6.1 CLI Commands

Command	Description
geoclaw-claude check	Detect if a newer version is available on GitHub
geoclaw-claude check --json	JSON output format
geoclaw-claude update	Pull latest code and reinstall (git pull + pip install -e .)
geoclaw-claude update --force	Force update even if already on latest version
geoclaw-claude update --test	Run tests after updating to verify stability
geoclaw-claude self-check	Full health check: modules, dependencies, config, environment
geoclaw-claude self-check --quick	Quick local check (skip remote version query)
geoclaw-claude self-check --json	JSON output format

6.2 Python API

```
from geoclaw_claude.updater import check, update, self_check,
print_self_check

check()                        # detect remote version
update()                       # git pull + pip install
print_self_check(self_check()) # print health report
```

Sample health report output:

```
╔══ GeoClaw-claude Self-Check ═══════════════════════╗
╟ Version      local: v2.2.1   remote: v2.2.1  ✓    ╟
╟ Modules      18/18 OK                              ╟
╟ Dependencies 12/12 OK                              ╟
╟ Config       API key: set  Data dir: OK            ╟
╚═══════════════════════════════════════════════════╝
```

7. Skill Script System

The Skill system allows users to define custom analysis workflows as Python scripts, with optional access to the Claude AI API through a standardized SkillContext interface.

7.1 Creating a Skill

```
# my_skill.py
SKILL_META = {
    "name": "my_analysis",
    "version": "1.0.0",
    "author": "your_name",
    "description": "My custom GIS analysis",
}

def run(ctx):
    layer     = ctx.get_layer("input")
    radius_km = float(ctx.param("radius_km", 5.0))
    ai_result = ctx.ask_ai("Analyze the spatial distribution of this
data")
    result    = do_analysis(layer, radius_km)
    return ctx.result(output=result)
```

7.2 CLI Commands

Command	Description
geoclaw-claude skill list	List all installed skills
geoclaw-claude skill run hospital_coverage --data hospitals.geojson --ai	Run a built-in skill with AI enabled
geoclaw-claude skill new my_analysis	Create a new skill template
geoclaw-claude skill install ./my_skill.py	Install a custom skill
geoclaw-claude skill info my_analysis	Show skill metadata and parameters

8. Other GIS Functions

8.1 Spatial Analysis

```
from geoclaw_claude.analysis.spatial_ops import buffer, nearest_neighbor,
kde, clip

buf     = buffer(hospitals, 1000, unit="meters")
nn      = nearest_neighbor(hospitals, metro_stations)
density = kde(hospitals, bandwidth=0.05, grid_size=100)
clipped = clip(hospitals, boundary)
```

8.2 Network Analysis

```
from geoclaw_claude.analysis.network import build_network, isochrone,
shortest_path

G    = build_network(bbox, network_type="drive")
iso  = isochrone(G, center=(114.30, 30.60), minutes=[5, 10, 15])
path = shortest_path(G, origin=(114.30, 30.60), destination=(114.40,
30.70))
```

8.3 Raster Analysis

```
from geoclaw_claude.analysis.raster_ops import load_raster, slope,
reclassify, zonal_stats

dem     = load_raster("dem.tif")
slp     = slope(dem)
reclass = reclassify(slp, [(0, 5, 1), (5, 15, 2), (15, 90, 3)])
stats   = zonal_stats(dem, zones_layer, stats=["mean", "max"])
```

8.4 Coordinate Transform

```
from geoclaw_claude.utils.coord_transform import transform_layer,
wgs84_to_gcj02

gcj_layer = transform_layer(hospitals, "wgs84", "gcj02")
bd_layer  = transform_layer(hospitals, "wgs84", "bd09")
```

8.5 Data I/O

```
from geoclaw_claude.io.vector import load_vector
from geoclaw_claude.io.osm import download_osm

layer = load_vector("hospitals.geojson")
osm   = download_osm("Wuhan", amenity="hospital")
```

9. About

9.1 UrbanComp Lab

GeoClaw-claude is developed and maintained by UrbanComp Lab, a research group focused on urban computing and geographic information science.

Item	Details
Website	https://urbancomp.net
GitHub	https://github.com/whuyao/GeoClaw_Claude
Contact	contact@urbancomp.net
License	MIT License
Copyright	© 2025 UrbanComp Lab

9.2 Third-Party Attributions

Package	Attribution
trackintel	ETH Zurich · mie-lab — Core mobility analysis algorithms — https://github.com/mie-lab/trackintel
osmnx	Boeing (2017) — OpenStreetMap network data access and analysis
geopandas	Core geospatial data processing framework
folium	Interactive Leaflet.js maps in Python